PYTHON KURSU • HAFTA 9 • GÜN 1

# OOP 2: Miras ve Kapsülleme

Ders 17 / 32 • 3 saat • VS Code + Python 3.12

**Bu dersin sonunda…**

- Miras ile bir sınıfı başka bir sınıftan türetebileceksiniz.
- super() ile üst sınıfın __init__'ini ve metotlarını kullanabileceksiniz.
- Metotları ezerek (override) çok biçimli (polimorfik) kod yazabileceksiniz.
- _ , __ ve @property ile verileri koruyabileceksiniz (kapsülleme).
- Miras ve içermeyi birlikte kullanan bir şirket bordrosu modeli kurabileceksiniz.

AÇILIŞ • 10 DK

# Bugünün zaman planı

| Saat | Süre | Bölüm |
|---|---|---|
| 0:00–0:10 | 10 dk | Açılış: hedefler, Ders 16'in tekrarı |
| 0:10–1:25 | 75 dk | Blok 1: miras nedir • class Alt(Ust) • isinstance • super() ve yeni öznitelikler • metot ezme • çok biçimlilik • sınıf hiyerarşisi |
| 1:25–1:40 | 15 dk | Mola |
| 1:40–2:55 | 75 dk | Blok 2: kapsülleme • _ ve __ • @property ve setter • hesaplanan özellikler • miras mı içerme mi • bordro mini projesi • alıştırmalar |
| 2:55–3:00 | 5 dk | Ödev ve gelecek ders |

AÇILIŞ • DERS 16'IN TEKRARI • 6 DK

# Dört soruda geçen ders

**SORU 1 • Hata nerede?**

```
class Urun:
    def etiket():
        return 'x'
```

**SORU 2 • Ne yazar?**

```
class Sayac:
    def __init__(self):
        self.n = 0
a, b = Sayac(), Sayac()
a.n += 1
print(a.n, b.n)
```

**SORU 3 • Ne görünür?**

```
print(defter)
# Urun'da __str__ yok
```

**SORU 4 • Neden tehlikeli?**

```
class Hesap:
    hareketler = []
    ...
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

# Ödev 1: Kitap ve Kitaplık

```python
class Kitaplik:
    def __init__(self):
        self.kitaplar = []

    def ekle(self, kitap):
        self.kitaplar.append(kitap)

    def en_iyiler(self, n=3):
        sirali = sorted(self.kitaplar,
                        key=lambda k: k.puan, reverse=True)
        return sirali[:n]
```

İÇERME

### Kitaplik → Kitap

Kitaplık bir kitap değildir; kitapları içerir (Sepet–Urun gibi).

YENİ

### lambda

sorted(..., key=lambda k: k.puan): her kitap için puanı döndüren kısa fonksiyon.

BLOK 1 • 0:10–1:25 • 75 DK

# Miras: ortak olanı bir kez yaz

15 dk
**Miras nedir?**

25 dk
**super() ve ezme**

25 dk
**Çok biçimlilik**

10 dk
**Hatalar**

BLOK 1 • MIRAS NEDIR? • 15 DK

# Miras: “bir … dır” ilişkisi

### Benzetme: aile benzerliği

Çocuk, ebeveynlerinden göz rengini, boyunu devralır ama kendine has özellikleri de vardır. Alt sınıf da üst sınıfın her şeyini devralır, üstüne kendi özelliklerini ekler ya da bazı davranışları değiştirir.

| Cümle | Miras |
| --- | --- |
| Yönetici bir çalışandır. | `Yonetici(Calisan)` |
| Stajyer bir çalışandır. | `Stajyer(Calisan)` |
| Kare bir şekildir. | `Kare(Sekil)` |
| Sepet bir ürün değildir. | `miras YOK (içerme)` |

BLOK 1 • MIRAS NEDIR? • 15 DK

# class Yonetici(Calisan): hiçbir şey yazmadan her şey hazır

Üst sınıf

```
class Calisan:
    unvan = "Çalışan"

    def __init__(self, ad, departman, maas):
        self.ad = ad
        self.departman = departman
        self.maas = maas

    def maas_hesapla(self):
        return self.maas

    def bilgi(self):
        return f"{self.ad} ({self.departman})"
```

Alt sınıf

```
class Yonetici(Calisan):
    pass

y = Yonetici("Ayşe Yılmaz",
             "Yazılım", 95000)
print(y.bilgi())
print(y.maas_hesapla())
```

ÇIKTI

```
Ayşe Yılmaz (Yazılım)
95000
```

BLOK 1 • MIRAS NEDIR? • 15 DK

# isinstance: “bu nesne şu türden mi?”

```
ali = Calisan("Ali Demir", "Yazılım", 62000)
ayse = Yonetici("Ayşe Yılmaz", "Yazılım", 95000)
print(isinstance(ayse, Yonetici), isinstance(ayse, Calisan))
print(isinstance(ali, Yonetici))
print(issubclass(Yonetici, Calisan), type(ayse).__name__)
```

```
ÇIKTI
True True
False
True Yonetici
```

Her yönetici bir çalışandır (True), ama her çalışan bir yönetici değildir (False).

BLOK 1 • SUPER() VE EZME • 25 DK

# Alt sınıfa yeni öznitelik: super().__init__

```python
class Yonetici(Calisan):
    unvan = "Yönetici"

    def __init__(self, ad, departman, maas, prim_orani=0.20):
        super().__init__(ad, departman, maas)
        self.prim_orani = prim_orani
        self.ekip = []

    def ekibe_ekle(self, calisan):
        self.ekip.append(calisan)
```

**super()**

“Üst sınıfım.” Ortak işi (ad, departman, maaş) üst sınıf yapar; alt sınıf yalnızca kendi eklerini yazar.

BLOK 1 • SUPER() VE EZME • 25 DK

# Yöneticiyi kullanmak: devralınan + yeni

```
ayse = Yonetici("Ayşe Yılmaz", "Yazılım", 95000)
ali = Calisan("Ali Demir", "Yazılım", 62000)
ayse.ekibe_ekle(ali)
print(ayse.ad, ayse.prim_orani, [c.ad for c in ayse.ekip])
```

```
ÇIKTI
Ayşe Yılmaz 0.2 ['Ali Demir']
```

DEVRALINAN

**ad, departman, maas**

super().__init__ kurdu.

YENİ

**prim_orani, ekip, ekibe_ekle()**

Yalnızca Yonetici’de var.

BLOK 1 • SUPER() VE EZME • 25 DK

# Metot ezme (override) ve super().metot()

```
class Yonetici(Calisan):
    ...
    def maas_hesapla(self):                # ezildi
        return self.maas * (1 + self.prim_orani)

    def bilgi(self):                       # genişletildi
        return super().bilgi() + f" · ekip: {len(self.ekip)} kişi"
```

| Yol | Ne yapar? | Örnek |
| --- | --- | --- |
| Tamamen değiştirmek | üst sınıfın metodu hiç çalışmaz | `maas_hesapla` |
| Genişletmek | super().metot() + ek iş | `bilgi` |

BLOK 1 • SUPER() VE EZME • 25 DK

# Aynı döngü, iki farklı sınıf

```
ali = Calisan("Ali Demir", "Yazılım", 62000)
ayse = Yonetici("Ayşe Yılmaz", "Yazılım", 95000)
ayse.ekibe_ekle(ali)
for kisi in [ali, ayse]:
    print(f"{kisi.unvan:<9}{kisi.bilgi():<40}{kisi.maas_hesapla():>10,.0f}")
```

```
ÇIKTI
Çalışan  Ali Demir (Yazılım)                          62,000
Yönetici Ayşe Yılmaz (Yazılım) · ekip: 1 kişi        114,000
```

Ali için Calisan'ın, Ayşe için Yonetici'nin metotları çalıştı.

BLOK 1 • ÇOK BIÇIMLILIK • 25 DK

# Üçüncü tür: Stajyer

```python
class Stajyer(Calisan):
    unvan = "Stajyer"

    def __init__(self, ad, departman, gunluk_ucret, gun_sayisi):
        super().__init__(ad, departman, maas=0)
        self.gunluk_ucret = gunluk_ucret
        self.gun_sayisi = gun_sayisi

    def maas_hesapla(self):
        return self.gunluk_ucret * self.gun_sayisi
```

FARK

**maas yerine günlük ücret**

super().__init__(..., maas=0); asıl hesap maas_hesapla'da.

ÖRNEK

**900 TL × 20 gün**

Stajyer("Can Öz", "Yazılım", 900, 20) → 18.000 (kurgusal)

BLOK 1 • ÇOK BIÇIMLILIK • 25 DK

# Çok biçimlilik: aynı çağrı, farklı davranış

**Benzetme**

“Ses çıkar!” dendiğinde kedi miyavlar, köpek havlar. Emir aynı, davranış nesneye göre farklı. Bordro döngüsü de kişiye “maaşını hesapla” der; nasıl hesaplanacağını kişinin sınıfı bilir.

```
toplam = 0
for kisi in personel:
    tutar = kisi.maas_hesapla()       # her nesne kendi metodu
    toplam += tutar
    print(f"{kisi.unvan:<9}{kisi.ad:<13}{tutar:>10,.0f}")
print(f"{'Toplam':<22}{toplam:>10,.0f}")
```

```
ÇIKTI
Çalışan   Ali Demir        62,000
Yönetici  Ayşe Yılmaz     114,000
Stajyer   Can Öz           18,000
Çalışan   Elif Ak          58,000
Toplam                    252,000
```

BLOK 1 • ÇOK BIÇIMLILIK • 25 DK

# Sınıf hiyerarşisi ve arama sırası

| Seviye | Sınıf | Neyi devralır? |
|---|---|---|
| 0 | `object` | Python'un en temel sınıfı: __str__, __init__ … |
| 1 | `Calisan` | object'ten |
| 2 | `Yonetici · Stajyer` | Calisan'dan (o da object'ten) |

```
Yonetici.__mro__ ve Calisan.__bases__
(<class '__main__.Yonetici'>, <class '__main__.Calisan'>, <class
  'object'>)
(<class 'object'>,)
```

**Arama sırası**

ayse.bilgi() → önce Yonetici'de, yoksa Calisan'da, yoksa object'te aranır. İlk bulunan çalışır.

BLOK 1 • SIK YAPILAN HATALAR

# Miras hataları

| Hatalı kod | Hata mesajı | Çözüm |
| --- | --- | --- |
| `def __init__(self, ad, prim): self.prim = prim # super yok` | `AttributeError: 'Yonetici' object has no attribute 'ad'` | super().__init__(ad) |
| `super().__init__() # ad verilmedi` | `TypeError: Calisan.__init__() missing 1 required positional argument: 'ad'` | super().__init__(ad) |
| `ali.ekibe_ekle(…) # ali bir Calisan` | `AttributeError: 'Calisan' object has no attribute 'ekibe_ekle'` | metot alt sınıfta |
| `class Yonetici(Calisan) # Calisan tanımsız` | `NameError: name 'Calisan' is not defined` | önce üst sınıf tanımlanır |

1:25–1:40 • 15 DK

# Mola

Moladan sonra: ayse.bakiye = 1_000_000 yazılmasını nasıl engelleriz?

BLOK 2 • 1:40–2:55 • 75 DK

# Kapsülleme ve mini proje

25 dk
**Kapsülleme**

8 dk
**Miras mı içerme mi?**

24 dk
**Bordro projesi**

18 dk
**Alıştırmalar**

BLOK 2 • KAPSÜLLEME • 25 DK

# Sorun: kurallar metotlardaydı, kapı ise açık

```
class BankaHesabi:
    def __init__(self, sahip, bakiye=0):
        self.sahip = sahip
        self.bakiye = bakiye

    def cek(self, tutar):
        if tutar > self.bakiye:
            raise ValueError("Yetersiz bakiye.")
        self.bakiye -= tutar


hesap = BankaHesabi("Ayşe", 1000)
hesap.bakiye = -5000          # hiçbir kural çalışmadı!
print(hesap.bakiye)
```

```
ÇIKTI
-5000
```

**Benzetme: ATM ve kasa**

Paraya ATM (metotlar) üzerinden ulaşırsınız; ATM kuralları uygular. Kasanın kapısı (öznitelik) herkese açık olsaydı kurallar anlamsız kalırdı.

BLOK 2 • KAPSÜLLEME • 25 DK

# _bakiye: “bu iç detaydır” işareti

```python
class BankaHesabi:
    def __init__(self, sahip, bakiye=0):
        self.sahip = sahip
        self._bakiye = bakiye        # _ : "içeriye ait, dokunma"

    def bakiye_goster(self):
        return self._bakiye


hesap = BankaHesabi("Ayşe", 1000)
print(hesap.bakiye_goster())
print(hesap._bakiye)                # mümkün ama kurala aykırı
```

```
ÇIKTI
1000
1000
```

**_ad**

## tek alt çizgi

Gelenek: “Sınıfın iç işi; dışarıdan kullanmayın.” Python engellemez; programcılar uyar.

BLOK 2 • KAPSÜLLEME • 25 DK

# __bakiye: adı gizlenen öznitelik

```
class BankaHesabi:
    def __init__(self, sahip, bakiye=0):
        self.sahip = sahip
        self.__bakiye = bakiye       # ad karıştırma

    def bakiye_goster(self):
        return self.__bakiye


hesap = BankaHesabi("Ayşe", 1000)
print(hesap.bakiye_goster())
try:
    print(hesap.__bakiye)
except AttributeError as hata:
    print("AttributeError:", hata)
print(hesap._BankaHesabi__bakiye)  # gerçek adı (kullanmayın!)
```

```
ÇIKTI
1000
AttributeError: 'BankaHesabi'
  object has no attribute
  '__bakiye'
1000
```

BLOK 2 • KAPSÜLLEME • 25 DK

# @property: öznitelik gibi okunan metot

```
class BankaHesabi:
    def __init__(self, sahip, bakiye=0):
        self.sahip = sahip
        self._bakiye = bakiye

    @property
    def bakiye(self):                # okunabilir, yazılamaz
        return self._bakiye

    def yatir(self, tutar):
        ...
        self._bakiye += tutar

hesap = BankaHesabi("Ayşe", 1000)
hesap.yatir(250)
print(hesap.bakiye)                  # parantez yok!
hesap.bakiye = -5000                 # try içinde
```

```
ÇIKTI
1250
AttributeError: property 'bakiye'
  of 'BankaHesabi' object has no
  setter
```

BLOK 2 • KAPSÜLLEME • 25 DK

# @maas.setter: atamayı kurala bağlamak

```
class Calisan:
    def __init__(self, ad, maas):
        self.ad = ad
        self.maas = maas              # burada da setter çalışır

    @property
    def maas(self):
        return self._maas

    @maas.setter
    def maas(self, yeni):
        if yeni < 0:
            raise ValueError("Maaş negatif olamaz.")
        self._maas = yeni

    @property
    def yillik_maas(self):            # hesaplanan, salt okunur
        return self._maas * 12
```

```
ali = Calisan("Ali Demir", 62000)
ali.maas = 65000
print(ali.maas,
      ali.yillik_maas)
ali.maas = -1   # try içinde
```

```
ÇIKTI
65000 780000
Reddedildi: Maaş negatif olamaz.
```

# Kapsülleme ve property hataları

| Hatalı kod | Hata mesajı | Çözüm |
|---|---|---|
| `hesap.bakiye() # property` | `TypeError: 'int' object is not callable` | parantezsiz: hesap.bakiye |
| `hesap.bakiye = 5 # setter yok` | `AttributeError: property 'bakiye' of 'H' object has no setter` | metotla değiştirin |
| `hesap.__bakiye` | `AttributeError: 'H' object has no attribute '__b'` | property ya da metot |
| `@property maas: return self.maas` | `AttributeError: property 'maas' of 'C' object has no setter` | return self._maas |

BLOK 2 • MIRAS MI IÇERME MI? • 8 DK

# Miras mı, içerme mi?

| | Miras (is-a) | İçerme (has-a) |
|---|---|---|
| Cümle | Yönetici bir çalışandır | Şirketin çalışanları vardır |
| Kod | class Yonetici(Calisan) | self.calisanlar = [] |
| Bağ | sıkı: üst sınıf değişirse alt etkilenir | gevşek: parçalar değiştirilebilir |
| Örnek | Stajyer(Calisan), Kare(Sekil) | Sepet → Urun, Sirket → Calisan |

**Kural**

Emin değilseniz içermeyi seçin. Miras yalnızca “bir … dır” cümlesi gerçekten doğruysa.

BLOK 2 • BORDRO PROJESI • 24 DK

# Mini proje: şirket bordrosu

| Sınıf | İlişki | Görevi |
|---|---|---|
| `Calisan` | üst sınıf | ad, departman, maaş; maas_hesapla(), bilgi() |
| `Yonetici` | Calisan'dan miras + ekibi içerir | primli maaş; ekibe_ekle() |
| `Stajyer` | Calisan'dan miras | günlük ücret × gün |
| `Sirket` | Calisan nesnelerini içerir | ise_al(), bordro(), departman_toplamlari() |

Bordro döngüsü kişinin türünü sormaz: **kisi.maas_hesapla()** yeter (çok biçimlilik).

BLOK 2 • BORDRO PROJESI • 24 DK

# Sirket sınıfı

departman_toplamlari .py dosyasında

```
class Sirket:
    def __init__(self, ad):
        self.ad = ad
        self.calisanlar = []

    def ise_al(self, *kisiler):
        self.calisanlar.extend(kisiler)

    def bordro(self, donem):
        print(f"{self.ad} · {donem} bordrosu")
        toplam = 0
        for kisi in self.calisanlar:
            tutar = kisi.maas_hesapla()          # çok biçimlilik
            toplam += tutar
            print(f"  {kisi.unvan:<9}{kisi.ad:<14}{tutar:>10,.0f} TL")
        print(f"  {'Toplam':<23}{toplam:>10,.0f} TL")
```

BLOK 2 • BORDRO PROJESI • 24 DK

# Bordroyu çalıştırınca

```
ayse.ekibe_ekle(ali)
ayse.ekibe_ekle(can)
sirket.ise_al(ayse, ali, can, elif_)
sirket.bordro("Ekim 2026")
print(sirket.departman_toplamlari())
print(ayse.bilgi())
```

```
ÇIKTI
Örnek Yazılım A.Ş. · Ekim 2026 bordrosu
  Yönetici Ayşe Yılmaz      114,000 TL
  Çalışan  Ali Demir         62,000 TL
  Stajyer  Can Öz            18,000 TL
  Çalışan  Elif Ak           58,000 TL
  Toplam                    252,000 TL
{'Yazılım': 194000.0, 'Pazarlama': 58000}
Ayşe Yılmaz (Yazılım) · ekip: 2 kişi
```

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 18 DK

# Alıştırma 1, 2 ve 3

KOLAY • 4 dk

### 1 • Şekiller

Sekil(ad) sınıfı alan() → 0 ve __str__; Daire(r) ve Kare(a) alan()'ı ezsin. Listedeki şekillerin alanlarını yazdırın.

ORTA • 6 dk

### 2 • Vadeli hesap

Bakiyesi salt okunur (property) BankaHesabi; VadeliHesap(BankaHesabi): faiz_ekle(oran) eklesin, cek() ise her zaman ValueError versin.

ZOR • 8 dk

### 3 • Zam ve rapor

Calisan.maas'a setter (negatif yok). Sirket'e zam_yap(departman, oran) ve en_yuksek() metotlarını ekleyin.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

# Çözüm 1: şekiller

```
class Sekil:
    def __init__(self, ad):
        self.ad = ad

    def alan(self):
        return 0

    def __str__(self):
        return f"{self.ad}: {self.alan():.2f}"

class Daire(Sekil):
    def __init__(self, r):
        super().__init__("Daire")
        self.r = r

    def alan(self):
        return math.pi * self.r ** 2
```

```
for sekil in [Daire(2), Kare(3),
              Sekil("Nokta")]:
    print(sekil)
```

```
ÇIKTI
Daire: 12.57
Kare: 9.00
Nokta: 0.00
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

# Çözüm 2: vadeli hesap

```
class VadeliHesap(BankaHesabi):
    def faiz_ekle(self, oran):
        self._bakiye += self._bakiye * oran

    def cek(self, tutar):
        raise ValueError("Vadeli hesaptan vade dolmadan çekim yapılamaz.")
```

```
ÇIKTI
10400.0
Reddedildi: Vadeli hesaptan vade
  dolmadan çekim yapılamaz.
```

```
vadeli = VadeliHesap("Ayşe", 10000)
vadeli.faiz_ekle(0.04)
print(vadeli.bakiye)
vadeli.cek(100)   # try içinde
```

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

# Çözüm 3: zam ve rapor

```
class Sirket:
    def __init__(self, calisanlar):
        self.calisanlar = calisanlar

    def zam_yap(self, departman, oran):
        for kisi in self.calisanlar:
            if kisi.departman == departman:
                kisi.maas = round(kisi.maas * (1 + oran))

    def en_yuksek(self):
        return max(self.calisanlar, key=lambda k: k.maas)
```

```
s.zam_yap("Pazarlama", 0.10)
enb = s.en_yuksek()
```

Calisan'daki maaş setter'ı slayt 21'deki ile aynı.

```
ÇIKTI
[('Ali Demir', 62000), ('Elif Ak', 63800), ('Deniz Kara', 67100)]
En yüksek: Deniz Kara 67100
```

KAPANIŞ • 5 DK

# Bugün ne öğrendik?

- **class Alt(Ust)**: alt sınıf üst sınıfın her şeyini devralır; “bir … dır” ilişkisi.
- Alt sınıfta __init__ yazınca **super().__init__(…)** unutulmaz.
- Aynı adlı metot **ezilir**; **super().metot()** ile genişletilir; aynı çağrı farklı davranış = çok biçimlilik.
- Kapsülleme: **_ad** gelenek, **__ad** ad karıştırma, **@property** salt okunur, **@x.setter** kontrollü atama.
- “Bir … dır” → miras; “… vardır” → içerme.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

# Ödev

1 • ZORUNLU

### Kütüphane üyeleri

Uye(ad) üst sınıfı; Ogrenci(Uye) en fazla 3, Ogretmen(Uye) en fazla 10 kitap alabilsin. odunc_al/iade_et metotları limiti kontrol etsin (sınıf değişkeni).

2 • ZORUNLU

### Kitap'a property

Ders 16 Kitap sınıfında puan'ı property + setter yapın (1–5 dışı ValueError); yas property'si yayın yılından kitabın yaşını hesaplasın.

3 • BONUS

### Oyun karakterleri

Karakter(ad, can) → Savasci, Buyucu, Okcu; her biri saldir(hedef)'i farklı uygulasın; can property'si 0'ın altına inmesin. Kısa bir dövüş simülasyonu yazın.

Teslim: ders17_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 9 • GÜN 2

# NumPy 1: sayısal diziler

Bir sınıfın 30 öğrencisinin notlarına 5 puan eklemek için döngü yazdık. Gelecek ders tek satırda ve çok daha hızlı:

- NumPy nedir, neden hızlı?
- np.array ve vektörel işlemler
- mean, max, argmax gibi metotlar
- matris oluşturma • not ve sıcaklık dizileri

Önizleme

```
import numpy as np

notlar = np.array([72, 45, 88, 91])
print(notlar + 5)
print(notlar.mean())
```

```
[77 50 93 96]
74.0
```

KAYNAKLAR

# Bu dersin konuları resmi belgelerde

| Ne için? | Adres |
| --- | --- |
| Python belgeleri: Miras | docs.python.org/3/tutorial/classes.html#inheritance |
| Özel değişkenler ve ad karıştırma | docs.python.org/3/tutorial/classes.html#private-variables |
| property belgesi | docs.python.org/3/library/functions.html#property |
| super() belgesi | docs.python.org/3/library/functions.html#super |
| Real Python: inheritance & composition | realpython.com/inheritance-composition-python |